

SymPy Bug Report

1 Bug 7: dsolve returns an invalid branch for f prime equals sqrt
f squared

1.1 Minimal Reproducer

```
from sympy import Eq, Function, N, diff, dsolve, simplify, sqrt, symbols

x = symbols("x")
f = Function("f")
C1 = symbols("C1")

ode = Eq(diff(f(x), x), sqrt(f(x)**2))
sol = dsolve(ode)

rhs = sol[0].rhs.subs(C1, 1)
residual = diff(rhs, x) - sqrt(rhs**2)

print("dsolve result:", sol)
print("chosen returned branch with C1=1:", rhs)
print("residual:", simplify(residual))
print("residual at x=0:", N(residual.subs(x, 0), 50))
```

1.2 Observed Behavior

```
dsolve result: [Eq(f(x), C1*exp(-x)), Eq(f(x), C1*exp(x))]
chosen returned branch with C1=1: exp(-x)
residual: -sqrt(exp(-2*x)) - exp(-x)
residual at x=0: -2.0000000000000000000000000000000000000000000000000000000
```

SymPy returns an unrestricted decreasing exponential branch. Setting the arbitrary constant in that branch to 1 gives $f(x) = e^{-x}$, but the residual of the original differential equation is nonzero. At $x = 0$, the left-hand side is -1 while the right-hand side is 1 .

1.3 Expected Behavior

The returned solution set should not include an unrestricted branch $f(x) = C_1 e^{-x}$. For real-valued solutions, any decreasing exponential branch must carry the sign condition $C_1 < 0$; equivalently, the solver should leave the branch-sensitive solution implicit or return a branch-conditioned answer. Positive-constant decreasing exponentials such as e^{-x} are not solutions of $f' = \sqrt{f^2}$.

1.4 Mathematical Explanation

For real-valued f ,

$$\sqrt{f(x)^2} = |f(x)|.$$

Therefore the differential equation is branch-sensitive:

$$f'(x) = |f(x)|.$$

On a positive branch, this reduces to $f' = f$, so the nonzero solutions are $f(x) = Ce^x$ with $C > 0$. On a negative branch, this reduces to $f' = -f$, so the nonzero solutions are $f(x) = Ce^{-x}$ with $C < 0$. The zero solution is also valid.

The returned expression C_1e^{-x} is not wrong merely because of a different representation or a missing integration constant; it is wrong because SymPy returns it with C_1 unrestricted. Substituting $C_1 = 1$ gives

$$f(x) = e^{-x}, \quad f'(x) = -e^{-x}, \quad \sqrt{f(x)^2} = \sqrt{e^{-2x}} = e^{-x}$$

on the real line, and hence

$$f'(x) - \sqrt{f(x)^2} = -2e^{-x},$$

which is not identically zero.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant source-level path in the ODE solver. In `sympy/solvers/ode/single.py`, the `Separable` hint builds an implicit separable solution equivalent to

$$\int^{f(x)} \frac{dy}{\sqrt{y^2}} = C_1 + \int^x 1 \, dx.$$

During simplification, `odesimp` in `sympy/solvers/ode/ode.py` calls `_handle_Integral`, which evaluates the left integral to a branch-sensitive expression involving $f(x)\log(f(x))/\sqrt{f(x)^2}$. The traced call path then reaches the explicit solving step in `odesimp`, reported around `sympy/solvers/ode/ode.py:1666`.

The key problematic rule is that `odesimp` explicitly solves the branch-sensitive implicit equation without retaining the sign condition that distinguishes the positive and negative real branches:

```
eq = _handle_Integral(eq, func, hint)
...
eqsol = solve(eq, func, force=True, rational=False if floats else None)
...
eq = [Eq(f(x), _expand(t)) for t in eqsol]
```

This produces both C_1e^{-x} and C_1e^x as explicit branches with an unrestricted integration constant. The decreasing exponential is valid only on a negative branch of f , so the unrestricted return value falsely permits positive constants. A plausible fix direction, supported by the diagnosis, is to avoid forcing branch-sensitive implicit ODE solutions into unconstrained explicit branches unless the necessary sign conditions can be attached. A post-solve residual validation step, similar in spirit to checking returned ODE solutions, would also reject the positive-constant decreasing branch.

1.6 Additional Failing Instances

The independent verification exercises the same returned branch with positive constants 1 through 6. In each case SymPy's unrestricted $C_1 e^{-x}$ branch admits $C_1 = c > 0$, producing $f(x) = ce^{-x}$. The residual at $x = 0$ is

$$f'(0) - \sqrt{f(0)^2} = -c - \sqrt{c^2} = -2c,$$

so every tested positive constant gives a concrete false solution. The expected behavior is the same across the family: the decreasing branch should either be absent for positive constants or be returned with a condition such as $C_1 < 0$.

Input / Expression	SymPy behavior	Expected behavior
$C_1 = 1$ in $C_1 e^{-x}$	admits e^{-x} , residual at 0 is -2	reject this branch or require $C_1 < 0$
$C_1 = 2$ in $C_1 e^{-x}$	admits $2e^{-x}$, residual at 0 is -4	reject this branch or require $C_1 < 0$
$C_1 = 3$ in $C_1 e^{-x}$	admits $3e^{-x}$, residual at 0 is -6	reject this branch or require $C_1 < 0$
$C_1 = 4$ in $C_1 e^{-x}$	admits $4e^{-x}$, residual at 0 is -8	reject this branch or require $C_1 < 0$
$C_1 = 5$ in $C_1 e^{-x}$	admits $5e^{-x}$, residual at 0 is -10	reject this branch or require $C_1 < 0$
$C_1 = 6$ in $C_1 e^{-x}$	admits $6e^{-x}$, residual at 0 is -12	reject this branch or require $C_1 < 0$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public material concerns the general branch/sign issue for $\sqrt{x^2}$, including SymPy documentation that $\sqrt{x^2}$ is not generally equal to x and general references noting that $\sqrt{x^2} = |x|$ over the reals. Focused searches did not identify a public report for this specific `dsolve` input, the returned invalid $C_1 e^{-x}$ branch, or the residual failure above. The deduplication verdict is therefore a new instance of a known failure mode: the broad square-root branch issue is known, but this exact ODE solver counterexample is previously unreported in the available evidence and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, Function, N, diff, dsolve, sqrt, symbols

@pytest.mark.parametrize("c", [1, 2, 3, 4, 5, 6])
def test_dsolve_sqrt_f_squared_branches_satisfy_ode_for_positive_constants(c):
    x = symbols("x")
    f = Function("f")
    ode = Eq(diff(f(x), x), sqrt(f(x)**2))
    sol = dsolve(ode)
```

```

for equation in sol:
    rhs = equation.rhs
    constants = [s for s in rhs.free_symbols if s != x]
    concrete_rhs = rhs.subs({constant: c for constant in constants})
    residual_at_zero = N((diff(concrete_rhs, x) - sqrt(concrete_rhs**2)).subs(x, 0),
50)
    assert abs(complex(residual_at_zero)) < 1e-40

```